pfi

# Python for Fixed Income

COURSE 2 · BOND MATH

# Cash Flows and Price From Yield

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# The desk lead's note: the coupons join the 100

## The Desk Lead's Note

*Notebook 01 valued each bond's 100 at maturity. That is not a price: Quorvane also pays ten coupons. List every payment each bond makes, price all four from their yields, and split each price into the 100 and the coupons. Put both steps in "bondmath", docstring first, and prove them against Excel's PV and PRICE. Tell me why clean and dirty are the same number today. Read the change before you save it, and take it home.*

The issuers are invented; every number describes the data.

### Start the Day

As in notebook 01, the setup makes a new work folder; this time it also holds the module, its tests, and notebook 01's commit.

```
work = start_day(2)
Work folder pfi_course2_02_k4w7q2xr: 2 data files, the module as notebook 01 left it, 1 commit
...   [100%]
3 passed in 0.50s
```

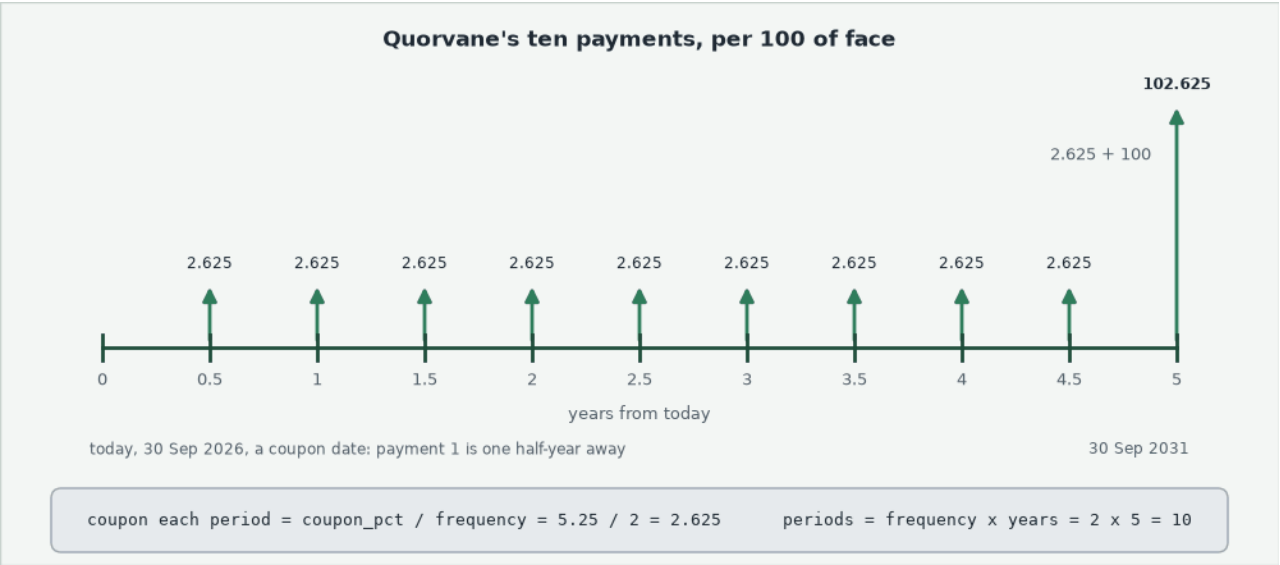
**The module starts the day as notebook 01 left it: two functions, three tests.**

### The Four Bonds

| issuer | ticker | coupon_pct | years | yield_pct |
|---|---|---|---|---|
| Quorvane Energy | QVNE | 5.250 | 5 | 5.400 |
| Dunmarrow Rail | DNMR | 6.125 | 7 | 6.125 |
| Pellwick Paper | PLWK | 4.750 | 3 | 4.600 |
| Halvering Foods | HLVF | 7.500 | 10 | 7.700 |

**76.6118 is Quorvane's 100 today; now the coupons join it.**

# 2.1 The cash flows of a bullet bond




A bullet bond pays a fixed coupon every period and repays all of its 100 on one day, at maturity. Quorvane's 5.25 percent a year arrives as 2.625 every half-year, and the 100 comes with the last coupon.

```
coupon = 5.25 / 2
rows = []
for k in range(1, 11):
    amount = coupon + 100 if k == 10 else coupon
    rows.append({'payment': k, 'time_years': k / 2, 'cash_flow': amount})
quorvane_flows = pd.DataFrame(rows)
quorvane_flows
```

The loop builds the flows Course 1's NumPy Part 2 built, and "x if test else y" is the one-line condition of Course 1's Introduction, Part 2.

## All the Cash

```
total_cash = quorvane_flows['cash_flow'].sum()
Total cash: 126.25 per 100 of face, 26.25 of it coupons
```

## Excel side by side

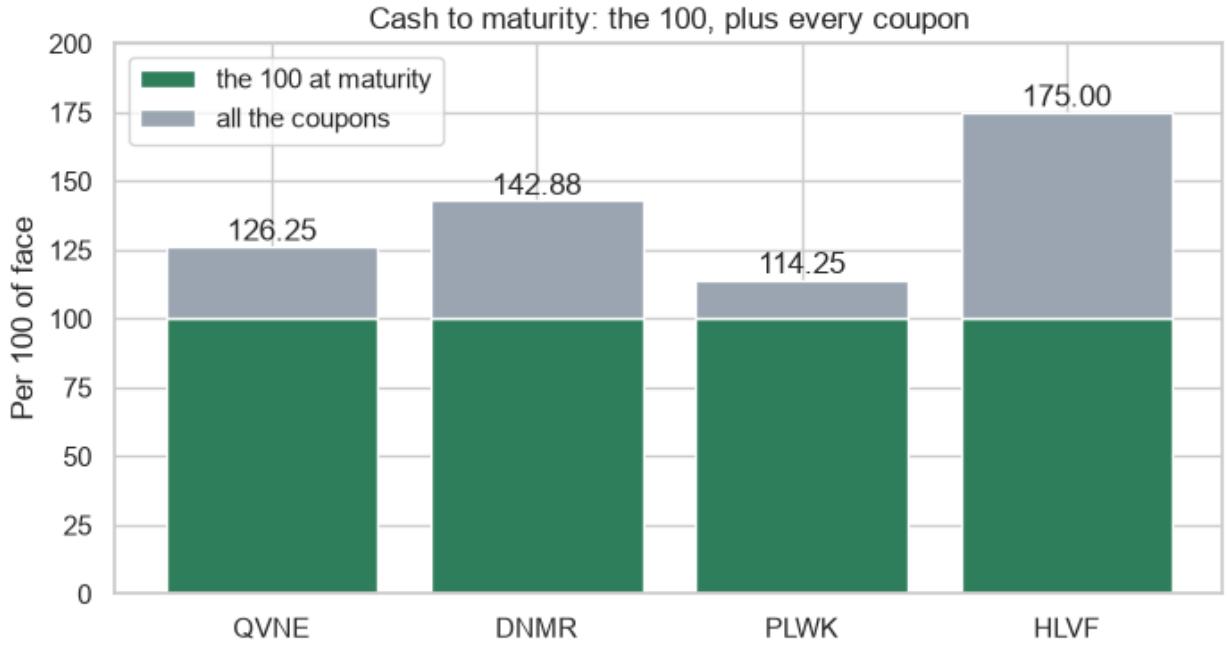
| Job | Python | Excel | Result |
|---|---|---|---|
| The payments | quorvane_flows | =5.25/2 in B1, filled to B9; =5.25/2+100 in B10 | 2.625, then 102.625 |
| All the cash | total_cash | =SUM(B1:B10) | 126.25 |

# Four bonds' payments, and all their cash

```
bonds['payments'] = 2 * bonds['years']
bonds['coupon_each'] = bonds['coupon_pct'] / 2
bonds['total_cash'] = bonds['payments'] * bonds['coupon_each'] + 100
```

| ticker | payments | coupon_each | total_cash |
|---|---|---|---|
| QVNE | 10 | 2.6250 | 126.25 |
| DNMR | 14 | 3.0625 | 142.88 |
| PLWK | 6 | 2.3750 | 114.25 |
| HLVF | 20 | 3.7500 | 175.00 |

A quarterly bond would split the same year's coupon four ways: more payments, the same total.




**Halvering pays 175.00 per 100 and Pellwick 114.25; what each is worth today is the next question.**

# 2.2 Price as the sum of discounted cash flows

Payment k waits k half-years, so its discount factor is 1 / 1.027 to the power k, notebook 01's factor for its own wait.

**Each payment, its own discount factor, then the sum**

| payment k | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| cash flow | 2.625 | 2.625 | 2.625 | 2.625 | 2.625 | 2.625 | 2.625 | 2.625 | 2.625 | 102.625 |
| 1 / 1.027 ** k | 0.973710 | 0.948111 | 0.923185 | 0.898914 | 0.875282 | 0.852270 | 0.829864 | 0.808047 | 0.786803 | 0.766118 |

### Each Payment Discounted, Then Added Up

```
period_yield = 5.4 / 100 / 2
quorvane_flows['discount_factor'] = 1 / (1 + period_yield) ** quorvane_flows['payment']
quorvane_flows['present_value'] = quorvane_flows['cash_flow'] * quorvane_flows['discount_factor']
quorvane_flows.round(6)
price = quorvane_flows['present_value'].sum()
print(f"Quorvane's price at 5.40 percent: {price:.4f} per 100 of face")
```

# Three routes to one price in Excel

### Excel side by side. Three routes to one price; "PRICE", the fourth, comes in 2.5.

| Job | Python | Excel |
|---|---|---|
| Each payment discounted | `present_value column` | 1 to 10 in A1:A10; =B1/(1+5.4/100/2)^A1 in C1, filled to C10 |
| Their sum | `price` | =SUM(C1:C10) |
| One function over the payments | | =NPV(5.4/100/2,B1:B10) |
| One formula | | =-PV(5.4/100/2,2*5,5.25/2,100) |
| The 100 alone | `redemption_pv` | =-PV(5.4/100/2,2*5,0,100) |
| The coupons alone | `coupons_pv` | =-PV(5.4/100/2,2*5,5.25/2,0) |

"NPV" discounts its first value by one full period: right on a coupon date, where the first coupon is a half-year away. PV's third argument is the payment each period, the coupon; its fourth the amount at the end.

### Excel's Answer

```
tvm = pd.read_csv('bondmath_excel_reference_tvm.csv', index_col='case_id')
quorvane_row = tvm.loc['tvm_pv_bond_13']
print(quorvane_row['excel_formula'], 'returned', quorvane_row['excel_value'])
```

### The 100 and the Coupons

```
redemption_pv = 100 * quorvane_flows['discount_factor'].iloc[-1]
coupons_pv = price - redemption_pv
print(f'The 100: {redemption_pv:.4f}. The coupons: {coupons_pv:.4f}')
```

# 2.3 cash_flows in your module, docstring first

```
%%writefile -a bondmath.py
def cash_flows(coupon_pct: float, years: float, frequency: int = 2) -> list[tuple[float, float]]:
    if frequency not in (1, 2, 4):
        raise ValueError(f"frequency must be 1, 2, or 4, not {frequency}")
    periods = years * frequency
    if periods <= 0 or abs(periods - round(periods)) > 1e-9:
        raise ValueError(f"{years} years is not a whole number of periods at frequency {frequency}")
    periods = round(periods)
    coupon = coupon_pct / frequency
    flows = []
    for k in range(1, periods + 1):
        time_years = k / frequency
        amount = coupon + 100 if k == periods else coupon
        flows.append((time_years, amount))
    return flows
```

Both go in docstring first, as in notebook 01: units, convention, Excel. It returns a list of (time in years, amount) pairs, tuples as in Course 1's Introduction, Part 1, and the hint "-> list[tuple[float, float]]" says so. It refuses a frequency other than 1, 2 or 4, and a term that is not a whole number of periods. "1e-9" is 0.000000001, a tolerance, because decimals are not exact, as Course 1's Debugging notebook showed with "0.1 + 0.2".

## Reload and Compare

```
import importlib
importlib.reload(bondmath)
flows = bondmath.cash_flows(5.25, 5)
assert flows == list(zip(quorvane_flows['time_years'], quorvane_flows['cash_flow']))
print(len(flows), 'pairs, the same as the table. First and last:', flows[0], flows[-1])
10 pairs, the same as the table. First and last: (0.5, 2.625) (5.0, 102.625)
```

## A Part Period Is Refused

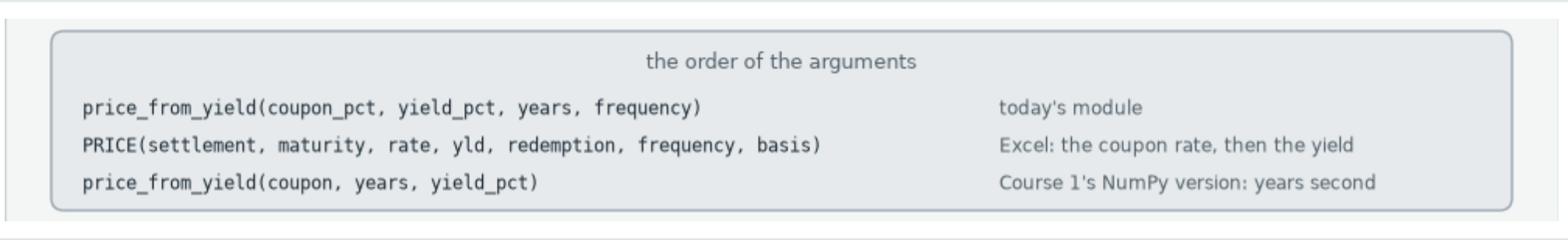
```
bondmath.cash_flows(5.25, 2.3)
ValueError: 2.3 years
is not a whole number of periods at frequency 2
```

A bond pays whole coupons, so the module refuses rather than return a number that looks right.

# price_from_yield: one function calls the other

It calls "cash_flows", then discounts each pair: "time_years * frequency" is the number of periods the payment waits. The arguments run coupon, yield, years, as Excel's "PRICE" takes the coupon rate before the yield; Course 1's NumPy version took years second.

```
%%writefile -a bondmath.py
def price_from_yield(coupon_pct: float, yield_pct: float, years: float, frequency: int = 2) -> float:
    period_yield = yield_pct / 100 / frequency
    price = 0.0
    for time_years, amount in cash_flows(coupon_pct, years, frequency):
        price += amount / (1 + period_yield) ** (time_years * frequency)
    return price
```




**The coupon rule lives in one place: "price_from_yield" never builds a payment itself.**

# 2.4 Excel's prices as tests

## A Test With a Filter

```
%%writefile -a test_bondmath.py
def test_price_from_yield_matches_excel():
    for row in tvm_rows("PV_BOND"):
        result = bondmath.price_from_yield(float(row["coupon_pct"]), float(row["rate_pct"]),
                                           float(row["years"]), int(row["frequency"]))
        assert result == pytest.approx(float(row["excel_value"]), abs=1e-6), row["case_id"]
    coupon_date_rows = [row for row in REFERENCE if row["coupdaybs"] == "0"]
    assert coupon_date_rows, "no coupon-date rows in the reference file"
    for row in coupon_date_rows:
        frequency = int(row["frequency"])
        years = int(row["coupnum"]) / frequency
        result = bondmath.price_from_yield(float(row["coupon_pct"]), float(row["yield_pct"]), years, frequency)
        assert result == pytest.approx(float(row["price"]), abs=1e-6), row["case_id"]
```

"REFERENCE" reads the dated answer key, "bondmath_excel_reference.csv": one row per bond and settlement date. The first test is a hand case: 6 percent for 5 years is ten coupons of 3.00, the last with the 100. "pytest.approx" compares a pair item by item. The second passes only if "cash_flows" refuses 2.3 years: "return" inside "except" ends the test there, and "assert False" fails it if no error came.

```
20 -PV rows, 1.0 to 30.0 years. By frequency: {1: 1, 2: 17, 4: 2}
17 PRICE rows on a coupon date. By basis: {0: 13, 1: 4}
```

The price test reads every "PV_BOND" row, then keeps the dated rows that settle on a coupon date: "coupdaybs", Excel's "COUPDAYBS", the days since the last coupon, is "0", in quotes because "read_rows" gives text. "coupnum", Excel's "COUPNUM", the coupons left, over the frequency is the years. "assert coupon_date_rows" stops a filter that found nothing: a test never passes by checking nothing.

```
......   [100%]
6 passed in 0.50s
```

**"6 passed": "price_from_yield" matches Excel's -PV on 20 rows and PRICE on the 17 that settle on a coupon date; 2.5 says why only those.**

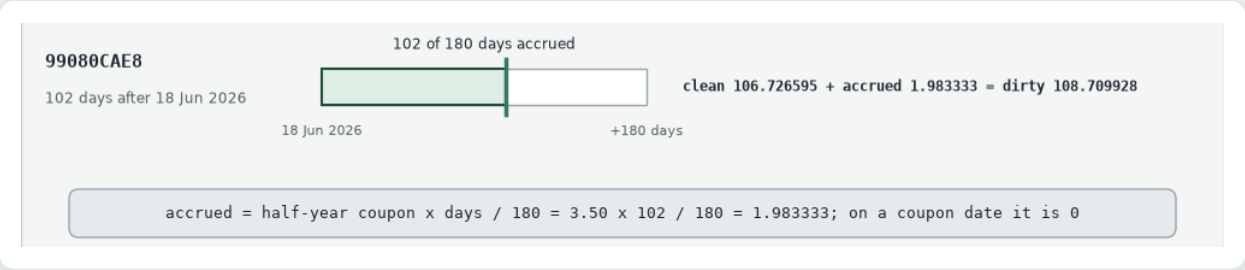
# 2.5 Clean equals dirty on a coupon date

Accrued interest is the part of the next coupon earned since the last one; the buyer pays it to the seller. The clean price leaves it out: the quoted price, and what Excel's "PRICE" returns. The dirty price is clean plus accrued: the cash that changes hands.

### The 17 Rows

```
Largest accrued on the 17 rows: 0.000000 per 100
```

On a coupon date nothing has accrued, so "price_from_yield" is both prices.




### A Bond Between Coupons

```
99080CAE8: last coupon 2026-06-18, 102 of 180 days ago
Clean 106.726595 + accrued 1.983333 = dirty 108.709928
```

Its accrued is the 3.50 half-year coupon times 102 / 180. Notebook 07 counts the days; notebook 08 prices a bond between coupons.

| Job | Python | Excel |
|---|---|---|
| Clean price from the dates | `bondmath.price_from_yield(5.25, 5.4, 5)` | =PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,5.4/100,100,2,0) |

PRICE's arguments, in order: settlement, maturity, rate (the coupon), yld, redemption, frequency, basis (0 is US 30/360), both rates as decimals.

**On a coupon date accrued is zero, so one number is both prices; between coupons they part, and notebook 08 prices that day.**

# 2.6 Read the change, commit, and bring it home

Read a change before you save it, whether it is yours, a colleague's, or an AI's. "git diff" shows every line that changed since the last commit, here notebook 01's.

```
!git -C "{work}" diff --stat
!git -C "{work}" diff bondmath.py
```

"log --oneline" lists the commits, newest first, one line each.

```
!git -C "{work}" add bondmath.py test_bondmath.py
!git -C "{work}" commit -m "Notebook 02: cash_flows and price_from_yield"
!git -C "{work}" log --oneline
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```

## Bring It Home

```
$work = "$env:TEMP\pfi_course2_02_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py" .
python -m pytest -q
git diff --stat
git add bondmath.py test_bondmath.py
git commit -m "Notebook 02: cash_flows and price_from_yield"
git log --oneline
```

Git then reports the commit, "Notebook 02: cash_flows and price_from_yield", and "git log --oneline" lists it above notebook 01's. In Colab, download the two ".py" files from the file browser on the left. The work folder can stay: a restart makes a new one.

**"my_bondmath" holds four functions, six tests, and two commits.**

# Predict first: write down your answer, then run the cell

## Predict 1

Notebook 01 put Quorvane's 100 at maturity at 76.6118 today. Add the ten coupons of 2.625, each discounted for its own wait. What is the bond worth today, per 100? Write down a number.

```
price = quorvane_flows['present_value'].sum()
print(f"Quorvane's price at 5.40 percent: {price:.4f} per 100 of face")
```

## Predict 2

Price Quorvane at a yield of 0.

A) 100: a bond starts from its face

B) 126.25: nothing is discounted

C) An error: the function cannot discount at a zero yield

```
print(bondmath.price_from_yield(5.25, 0, 5))
```

## Predict 3

Double Quorvane's coupon to 10.50, same yield and years.

A) About 198.70: the price doubles

B) About 125.60: the extra 26.25 of coupons, in full

C) About 122.09: the extra coupons, discounted

```
doubled = bondmath.price_from_yield(10.5, 5.4, 5)
print(f'Coupon 10.50: {doubled:.4f}. Its coupons: {doubled - redemption_pv:.4f}')
```

## Predict 4

"%%writefile -a" has already saved both files to disk. What will "git diff --stat" show?

A) Nothing: the files are saved

B) Lines added, and none removed

C) Every line of both files, removed and written again

```
!git -C "{work}" diff --stat
```

# The note, answered

| The note asked | Answer |
|---|---|
| Every payment | 10, 14, 6 and 20 payments; 114.25 to 175.00 per 100 in all |
| In the module, docstring first | cash_flows, price_from_yield |
| Against Excel | 6 passed, 37 rows of -PV and PRICE |
| Clean and dirty | the same today: accrued is 0 |
| Read, saved, taken home | the diff read, the second commit, my_bondmath |

Be proud: your module now prices a bond the way Excel's "PRICE" does, and you read every line before you saved it.

## You Can Now

List a bullet bond's payments: the coupon split by the frequency, and the 100 with the last.

Price a bond on a coupon date as the sum of its discounted payments, and split the price into the 100 and the coupons.

Get the same price in Excel four ways: a discounted column, "NPV", "-PV" with the coupon, and "PRICE".

Write a function that returns a list of tuples, and one that calls another.

Test against Excel with a filter that must find rows.

Say why clean equals dirty on a coupon date.

Read a change with "git diff" before you commit it.

**Next, The Price-Yield Relationship: why a bond prices above or below par, and how far it moves when its yield moves.**